1. Вложенные правила

В хорошо структурированных таблицах стилей нет необходимости присваивать каждому элементу классы. Достаточно лишь более подробно описывать стили элементов, используя возможность вкладывать селекторы в другие селекторы. К слову, такие селекторы называются вложенными и представляют собой объёмную структуру.

01. Пример вложения

Результат выполнения кода CSS:

```
#header .menu {
    color: red;
}
#header .logo {
    width: 100px;
}
```

Пример записи в less

```
#header{
    .menu{
        color: red;
    }
    .logo{
        width: 100px;
    }
}
```

2. Ссылка на родителя селектора

Во вложенной конструкции можно ссылаться на родительский класс, используя знак &. С помощью этого символа & можно обращаться к родителю текущего

селектора, а также сшивать, объединять их и создавать внутри них области видимости

02. Пример со ссылкой на родителя

Результат выполнения кода css:

```
a {
      color: blue;
}
a:hover {
      color: red;
}
```

Пример записи в less

```
a{
      color: blue;
        &:hover{
            color: red;
        }
}
```

3. прямое объединение селекторов

Родительский селектор может использоваться для объединения любых селекторов, псевдоэлементов и псевдоклассов.

03. Пример объединения .class-1 и .class-2.

Результат выполнения кода css:

```
.class-1 {
  background-color: #fff;
}
.class-1.class-2 {
  color: #000;
}
```

Пример записи в less

```
.class-1 {
  background-color: #fff;

  &.class-2 {
    color: #000;
  }
}
```

4. Обратное объединение селекторов

Во время компиляции, селектор, у которого имеется ссылка на родителя справа от имени, будет опущен ниже по древу вложенности вне зависимости от глубины вложенности. То есть при обратном объединении родительский селектор всегда ссылается на корневой селектор всего объявления.

Результат выполнения кода css:

```
.item-card {

  background-color: #f5f5f5;

  border: 1px solid green;

}

.main .item-card {

  background-color: #f5f5f5;

  border: 1px solid green;

}

.none-border .main .item-card {

  border: 0px;

}
```

Пример записи в less

```
.main {

  .item-card {

    background-color: #f5f5f5;

    border: 1px solid green;

    .none-border & {

      border: 0px;

    }

  }

}
```

5. Сшивание (склеивание) селекторов

Не редко требуется производить операцию склеивания имён текущего и родительского селектора. Такая практика применяется при создании новых классов на основе старого. Кроме того, такая операция просто необходима тем людям, кто использует методологию БЭМ (и ей подобные) при написании стилей.

04. Пример склеивание селекторов

Результат выполнения кода css:

Изначально кнопки имеют серый цвет.

```
.button {

  background-color: #ddd;
  color: #000;
}
```

Кнопка, предназначенная для добавления новой записи имеет зелёный цвет

```
.button-add {
  background-color: green;
  color: #fff;
}
```

Кнопка, предназначенная для удаления — красныйцвет.

```
.button-remove {
  background-color: red;
  color: #fff;
}
```

С помощью препроцессора этот код можно структуризировать, и тем самым упростить. Для этого используется сшивание селекторов. В этом случае наш код преобразуется таким образом:

Пример записи в less

```
.button {
  background-color: #ddd;
  color: #000;

  &-add {
    background-color: green;
    color: #fff;
  }

  &-remove {
    background-color: red;
    color: #fff;
  }
}
```

6. **Переменные упрощают разработку и поддержку кода, позволяя изменять значения из одного места в программе**

В любом полноценном языке программирования есть **переменные** (от англ. variables), которые представляют собой именованную область памяти и позволяют записывать в себя какие-либо данные. Переменные дают возможность обращаться к себе по имени, а также изменять присвоенные им значения.

Процесс объявления (определения) переменных и присваивания им значения называют *инициализацией*.

В Less переменные динамические(можно изменять тип), переопределяемые (можно менять значение) и требующие инициализации.

6.1. Переменные значений

Создание переменной начинается с символа **@,** далее без пробелов следует имя переменной, состоящее из латинских букв, цифр, тире и знака подчеркивания.

Создание (инициализация) переменной

```
// main: styles.less;
@textColor: #eee;
@bgColor: #000;

@grey: #eee;

@colorAccent: #3CFFFF;
@colorSecond: #26ADFF;
@colorDanger: #FC3F4A;
@lightGrey: #8f9496;


@linkColor: @colorSecond;
```

Пример кода:

```
@color: #eee;
@linkColor: #8f9496;

.block{
	background-color:@color; // вызов переменной
	color: @textColor;
}
a{
```

```
        color: @linkColor
}
```

Арифметические операции (обычный режим)

Допускается производить операции с выражениями, значения которых соответствуют строкам.

Пример	Название	Результат	Числа	Строки
-@a	Отрицание	Смена знака @a	+	ошибка
@a + @b	Сложение	Сумма @a и @b	+	игнорируется*
@a - @b	Вычитание	Разность @a и @b	+	игнорируется*
@a * @b	Умножение	Произведение @a и @b	+	игнорируется*
@a / @b	Деление	Частное от деления @a на @b	+	игнорируется*

```
@fontSize: 16px;
@h1: @fontSize*1.6;
@h2: @fontSize*1.4;
@h3: @fontSize*1.2;

@pad1: 5vw;
@pad2: 5px;
```

Пример

```
h1 {font-size: @h1};
```

Результат выполнения кода CSS:

```
h1 {font-size: 32px};
```

6.2. Переменные «Селекторы»

Переменная может содержать имя класса или идентификатора

Пример

```
/*переменная — имя селектора*/
@selector-class: border;
@selector-id: nav;

.@{selector-class}
 { border: 5px dotted @color; }

#@{selector-id}
{ padding: 10px; }


 //использование

.product{
    .border;
}
```

Результат выполнения кода CSS:

```
.border {border: 5px dotted @color;}

#nav{ padding: 10px; }
.product{
     border: 5px dotted @color;
}
```

Использование HTML:

```
<div class=”border”>
```

6.3. Переменные «Свойства»

```
/*переменная — имя свойства*/

@prop: color;
@selector-id: nav;
```

```
#@{selector-id} {
        @{prop}: pink;
        padding: 10px;
}
```

Результат выполнения кода CSS:

```
#nav{
   color:  pink;
   padding: 10px;
}
```

7. Миксины (примеси)

Препроцессор Less позволяет выполнять примеси — добавлять существующие свойства к вновь создаваемым свойствам. Этот инструмент существенно упрощает написание кода путем исключения повторного написания стилей

Если вы не хотите, чтобы сама примесь не присутствовала в коде CSS после компиляции, то необходимо поставить скобки после определения примеси

Примесь (от англ. mix-in) — набор свойств и селекторов, расширяющий поведение другой сущности (селектора).

Пример css

```
.bordered() {
  border-top: dotted 1px #333;
  border-bottom: solid 2px #333;
}
```

Использование

```
article {
  .bordered;
  color: #443d3d;
}
```

После компиляции селектор `.bordered` безвозмездно отдаст все свои свойства и вложенные правила селектору `article`. При этом в конечном CSS-файле будут объявлены оба этих селектора.

Реализация:

```
article {
  border-top: dotted 1px #333;
  border-bottom: solid 2px #333;
  color: #443d3d;
}
```

при компиляции не будет создан класс `.bordered`, так как у него указаны скобки после имени. Такая конструкция говорит компилятору, что она чистейшая примесь, которая не хочет быть скомпилирована без явных на то причин.

7.1. Примеси с параметрами

Для того, чтобы примеси были переиспользуемыми в различных контекстах, у них могут быть указаны параметры, в зависимости от которых может меняться цвет, фон и другие значения. Параметры указываются в скобках после имени примеси и представляют собой обычные локальные переменные.

```
.bordered(@color) {
  border-top: dotted 1px @color;
  border-bottom: solid 2px @color;
}

article {
  .bordered(#ccc);
  color: #443d3d;
}
```

Как результат

```
.article {
  border-top: dotted 1px #cccccc;
  border-bottom: solid 2px #cccccc;
  color: #443d3d;
}
```

7.2. Значения параметров по умолчанию

Важной отличительной чертой примесей является возможность указывать значения по умолчанию для переменных. То есть в случае, если примесь вызвана, а значения для параметров не были переданы или переданы частично, то ошибки компилятор не выдаст, а возьмёт значение, указанное по умолчанию.

```
.bordered(@color: #ccc) {
  border-top: dotted 1px @color;
  border-bottom: solid 2px @color;
}

article {
  .bordered();
  color: #443d3d;
}
```

Как результат

```
article {
  border-top: dotted 1px #cccccc;
  border-bottom: solid 2px #cccccc;
  color: #443d3d;
}
```

Пример с несколькими переменными

```
.flex(@dir, @con){
      display: flex;
     flex-direction: @dir;
     justify-content: @con;
}


.menu{
      .flex(row,space-between);
}
```

7.3. Специальные параметры

Кроме обычных локальных переменных для примеси, можно использовать специальные параметры, которые имеют несколько другое предназначение.

Переменная `@arguments`

Переменная `@arguments` представляет собой синоним для всех переданных в примесь значений при её вызове. Такое поведение может быть полезно при использовании примеси с большим количеством переменных, предназначающихся для одного свойства.

```
 .box-shadow(@x: 0, @y: 0, @blur: 1px, @color: #333) {
            box-shadow: @arguments;
     }

.big-block {
        .box-shadow(2px, 5px);
       }
```

результат

```
.big-block {
       box-shadow: 2px 5px 1px #333;
       }
```

Переменная `@rest`

Переменная, позволяющая указывать переменное количество передаваемых параметров. Звучит сложно, но на деле все проще:

```
.mixin(@width: 152px, @height: 20px, @rest...)  {
  width: @width;
  height: @height;
  border: @rest;
}

.class {
  .mixin(304px, 40px, 2px, solid, #ddd);
}
```

Обратите внимание на троеточие (`...`) после переменной `@rest`. Если бы его не было, то переменная `@rest` являлась бы обычной переменной со всеми вытекающими отсюда проблемами в виде ошибки компилятора.

Первые два значения будут присвоены переменным `@width` и `@height` соответственно, а остальные значения будут доступны переменной `@rest`.

```
.class {
  width: 304px;
  height: 40px;
```

```
  border: 2px solid #dddddd;
}
```

Переменные `@rest` и `@arguments` можно использовать вместе:

```
.mixin(@width: 152px, @height: 20px, @rest...)  {
  content: @arguments;
}

.class {
  .mixin(304px, 40px, 2px, solid, #ddd);
}
```

В этом случае свойство `content` получит все переданные переменные:

```
.class {
  content: 304px 40px 2px solid #dddddd;
}
```

Как и в случае с `@arguments`, используется крайне редко, так как такое поведение примесей избыточно и практически не имеет места применения.

Примеси как функции

Функция — представляет из себя именованную часть кода программы, к которой можно обратиться из другого места программы.

```
.btn-style(@status){
        &__@{status}{
                 background: @@status;
                & when (lightness(@@status)>80){color: green;}
                &:hover{
                .lighBG(@@status);
           }
        }
}
.btn{
```

```
  .btn-style(error);
  .btn-style(succes);
}
```